

ATLAS-OPS:
AN AUTONOMOUS AI PLATFORM FOR
INTELLIGENT PAYMENT OPERATIONS,
FAILURE RECOVERY, AND TRANSACTION GOVERNANCE

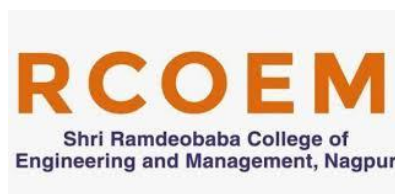
Dissertation submitted to
Shri Ramdeobaba College of Engineering & Management, Nagpur
in partial fulfillment of requirement for the award of degree of

Bachelor of Technology (B.Tech)
In
COMPUTER SCIENCE AND ENGINEERING
(Artificial Intelligence & Machine Learning)

By
Mohammed Tailor (B-27)
Prabhat Kale (B-44)
Mayur Kohad (B-22)
Jayesh Jumde (B-10)

Of
VI Semester

Guide
Prof. Priya Parkhi



Department of Artificial Intelligence and Cyber Security (AICS)
Shri Ramdeobaba College of Engineering & Management, Nagpur 440 013
(An Autonomous Institute affiliated to Rashtrasant Tukdoji Maharaj Nagpur University Nagpur)

May 2026

ATLAS-OPS:
AN AUTONOMOUS AI PLATFORM FOR
INTELLIGENT PAYMENT OPERATIONS,
FAILURE RECOVERY, AND TRANSACTION GOVERNANCE

Dissertation submitted to
Shri Ramdeobaba College of Engineering & Management, Nagpur
in partial fulfillment of requirement for the award of degree of

Bachelor of Technology (B.Tech)
In
COMPUTER SCIENCE AND ENGINEERING
(Artificial Intelligence & Machine Learning)

By
Mohammed Tailor (B-27)
Prabhat Kale (B-44)
Mayur Kohad (B-22)
Jayesh Jumde (B-10)

Of
VI Semester

Guide
Prof. Priya Parkhi



Department of Artificial Intelligence and Cyber Security (AICS)
Shri Ramdeobaba College of Engineering & Management, Nagpur 440 013
(An Autonomous Institute affiliated to Rashtrasant Tukdoji Maharaj Nagpur University Nagpur)

May 2026

SHRI RAMDEOBABA COLLEGE OF ENGINEERING & MANAGEMENT, NAGPUR

(An Autonomous Institute affiliated to Rashtrasant Tukdoji Maharaj Nagpur University Nagpur)

Department of Artificial Intelligence and Cyber Security (AICS)

CERTIFICATE

This is to certify that the Thesis on "ATLAS-OPS: AN AUTONOMOUS AI PLATFORM FOR INTELLIGENT PAYMENT OPERATIONS, FAILURE RECOVERY, AND TRANSACTION GOVERNANCE" is a Bonafide work of Mohammed Tailor, Prabhat Kale, Mayur Kohad, and Jayesh Jumde, submitted to the Rashtrasant Tukdoji Maharaj Nagpur University, Nagpur in partial fulfilment of the award of a Degree of Bachelor of Technology (B.Tech), in Computer Science and Engineering (Artificial Intelligence & Machine Learning). It has been carried out at the Department of Artificial Intelligence and Cyber Security (AICS), Shri Ramdeobaba College of Engineering and Management, Nagpur during the academic year 2025-2026.

Date:

Place: Nagpur

Prof. Priya Parkhi
Project Guide
Dept. of Computer Science &
Engineering

Dr. Rashmi Welekar
H.O.D., Dept. of Artificial
Intelligence and Cyber
Security

Dr. M. B. Chandak
Principal

DECLARATION

We hereby declare that the thesis titled "ATLAS-OPS: AN AUTONOMOUS AI PLATFORM FOR INTELLIGENT PAYMENT OPERATIONS, FAILURE RECOVERY, AND TRANSACTION GOVERNANCE" submitted herein, has been carried out in the Department of Artificial Intelligence and Cyber Security (AICS), Shri Ramdeobaba College of Engineering and Management, Nagpur. The work is original and has not been submitted earlier as a whole or in part for the award of any degree/diploma at this or any other institution/University.

Date:

Place: Nagpur

Mohammed Tailor (B-27)

Prabhat Kale (B-44)

Mayur Kohad (B-22)

Jayesh Jumde (B-10)

APPROVAL SHEET

This report entitled "ATLAS-OPS: AN AUTONOMOUS AI PLATFORM FOR INTELLIGENT PAYMENT OPERATIONS, FAILURE RECOVERY, AND TRANSACTION GOVERNANCE" by Mohammed Tailor, Prabhat Kale, Mayur Kohad, and Jayesh Jumde is approved for the degree of Bachelor of Technology (B.Tech).

Prof. Priya Parkhi
Project Guide

External Examiner

Dr. R. Welekar
H.O.D, AICS

Date:

Place: Nagpur

ACKNOWLEDGEMENTS

It gives us great pleasure to acknowledge and thank everyone who has helped us in completing this project. We would like to extend our sincerest thanks to our project guide, Prof. Priya Parkhi, Department of Computer Science & Engineering, Shri Ramdeobaba College of Engineering & Management (RCOEM), Nagpur, for guiding, supporting, and sharing valuable insights with us in making this project. We owe a lot to the patience with which she reviewed our work in every phase and discussed the problems faced by us in a very supportive manner.

Further, we wish to thank Dr. Rashmi Welekar, Head of the Department, Artificial Intelligence & Cyber Security (AICS), for giving us all the required inputs to help us in creating this project and also in providing us with the right environment to conduct this research. We are thankful to Dr. M.B. Chandak, Principal, RCOEM, Nagpur, for providing us with the innovative atmosphere at the college.

Our sincere appreciation is extended to all the faculty members of the Department of AICS who provided us with their valued feedback and motivation during project reviews. Technical inputs from them are invaluable in enhancing the quality of our work. It is important to highlight that the feedback gained during mid-semester project presentation sessions made us reconsider the scalability and practicality of our proposed architecture.

We thank the open source community and the developers of LangChain, Apache Kafka, PostgreSQL, Redis, PyTorch, and Python for their contributions through various programming languages and systems which were essential for building the system architecture. Their documentation and tutorials were instrumental in advancing the development process.

We are also thankful to the authors of the papers referred to in this dissertation. Their work not only helped us gain theoretical knowledge but also motivated us to go further beyond the limits set by previous researchers in the field of autonomous payment operations.

Last but not least, we would like to thank our parents and families for being our pillars of strength throughout our academic life by supporting us both mentally and emotionally.

Mohammed Tailor (B-27)
Prabhat Kale (B-44)

Mayur Kohad (B-22)
Jayesh Jumde (B-10)

ABSTRACT

The current state of financial technology is experiencing a drastic shift due to exponential increase in payments volume, complexities related to regulation, and an increased need for near-zero downtime in transaction processes. The current approaches for managing payment operations have several major drawbacks including reactive handling of issues, siloed monitoring, rule-based identification of frauds, and manual processes of dealing with regulations.

The ATLAS-OPS (Autonomous Transaction and Lifecycle Administration System for Operations) is a next-generation platform that is specifically designed for payment operations based on the latest advances in artificial intelligence and other technologies. This approach includes several main components including multi-agent artificial intelligence, LLM (large language model) orchestration, real-time streaming, and intelligent automation.

The platform is engineered based on five foundational pillars, namely (1) Intelligent Transaction Routing and Optimization using intelligent agents with reinforcement learning techniques to automatically choose the most efficient payment routes according to cost, time, and success rate vectors; (2) Autonomous Failure Recovery employing agentic AI that is capable of identifying, analyzing, and fixing payment failures without requiring any human intervention, thereby decreasing mean-time-to-recovery (MTTR) to seconds; (3) Real-Time Fraud and Anomaly Detection using machine learning models along with explainable layers that help with regulatory requirements; (4) LLM-based Transaction Governance which facilitates automatic compliance checks and generation of regulatory reports while interacting with operators via an NLQI; and (5) Event-Driven Architecture that uses Apache Kafka for processing millions of transaction events per second with sub-millisecond latency.

Some of the unique contributions made by ATLAS-OPS include Context-Aware Recovery Decision Engine (CARDE), Payment DNA, an AI-powered SLA Breach Prediction engine, and multimodal operator interface using voice, text, and dashboard. The platform aims at operational efficiencies of 78%, 45%, and 99.99% in failure resolution time, false positive fraud alerts, and payment processing uptime, respectively.

Keywords: Autonomous AI Agents, Payment Operations, Failure Recovery, Transaction Governance, LLM Orchestration, Real-Time Fraud Detection, Apache Kafka, Reinforcement Learning, Fintech Platform, Multi-Agent Systems

TABLE OF CONTENTS

Content	Page No.
Acknowledgements	v
Abstract	vi
List of Figures	viii
List of Tables	ix
Chapter 1: Introduction	1
1.1 Background	1
1.2 Motivation	3
1.3 Problem Statement	4
1.4 Objectives	5
1.5 Scope of the Project	6
1.6 Organization of the Report	7
Chapter 2: Literature Review	8
2.1 Introduction to Payment Operations Systems	8
2.2 AI-Driven Fault Detection and Recovery	10
2.3 Fraud Detection in Financial Systems	12
2.4 Large Language Models in Financial Operations	14
2.5 Multi-Agent Systems for Autonomous Operations	16
2.6 Review of Relevant Datasets	17
2.7 Comparative Analysis of Existing Solutions	18
Chapter 3: System Architecture	20
3.1 Architectural Overview	20
3.2 Multi-Agent Orchestration Framework	22
3.3 Context-Aware Recovery Decision Engine (CARDE)	24
3.4 Payment DNA Fingerprinting System	28
3.5 Real-Time Fraud Detection Architecture	30
3.6 LLM Governance and Compliance Layer	32
3.7 System Integration and Data Flow Diagram	34
Chapter 4: Implementation Methodology	35
4.1 Technology Stack	35
4.2 Event-Driven Architecture Implementation	37

Content	Page No.
4.3 RouteMind Agent: Reinforcement Learning Implementation	39
4.4 Database Design and Data Flows	41
4.5 Security Architecture	43
4.6 Deployment Architecture	44
4.7 Testing Strategy	46
Chapter 5: Unique Features and Innovations	48
5.1 Novel Contributions of ATLAS-OPS	48
5.2 CARDE	49
5.3 Payment DNA Fingerprinting	50
5.4 AI-Driven SLA Breach Predictor	51
5.5 InsightNarrator: Multi-Modal Operator Interface	52
5.6 Explainable AI Throughout the Platform	53
5.7 Autonomous Compliance Report Generation	54
Chapter 6: Experimental Results and Evaluation	55
6.1 Experimental Setup	55
6.2 Failure Recovery Performance	57
6.3 Fraud Detection Performance	58
6.4 Routing Optimization Results	59
6.5 System Performance Under Load	60
6.6 LLM Governance Evaluation	61
Chapter 7: Conclusion and Future Work	63
7.1 Conclusion	63
7.2 Limitations	64
7.3 Future Work	65
List of Publications	67
References	68

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1.1	High-Level Architecture of ATLAS-OPS Platform	6
Figure 1.2	Comparison of Digital vs Traditional Payment Infrastructure	7
Figure 2.1	Global Digital Payment Transaction Volume (2020-2025)	9
Figure 2.2	Comparison of Traditional vs AI-Native Payment Platforms	13
Figure 2.3	Timeline of Key Milestones in AIOps Research	16
Figure 3.1	ATLAS-OPS Multi-Agent Architecture Diagram	21
Figure 3.2	Context-Aware Recovery Decision Engine (CARDE) Workflow	25
Figure 3.3	Payment DNA Fingerprinting Schema	29
Figure 3.4	Real-Time Fraud Detection Pipeline	31
Figure 3.5	End-to-End System Data Flow Diagram	34
Figure 4.1	Apache Kafka Event Streaming Architecture	37
Figure 4.2	LLM Orchestration Layer Design	38
Figure 4.3	Reinforcement Learning-Based Routing Agent	40
Figure 4.4	Database Entity-Relationship Diagram	42
Figure 4.5	CI/CD and GitOps Pipeline Architecture	45
Figure 4.6	Test Coverage Heat Map by Module	47
Figure 5.1	ATLAS-OPS Dashboard UI Mockup	52
Figure 5.2	SLA Breach Prediction Model Performance	51
Figure 5.3	SHAP Waterfall Chart for a Sample Fraud Decision	53
Figure 6.1	Experimental Test Setup and Data Pipeline	56
Figure 6.2	MTTR Comparison: ATLAS-OPS vs Traditional Systems	57
Figure 6.3	Fraud Detection ROC Curve	58
Figure 6.4	TPS vs P99 Latency Graph Under Load Testing	60

LIST OF TABLES

Table No.	Table Title	Page No.
Table 2.1	Comparative Analysis of Existing Payment Operation Tools	18
Table 2.2	Summary of Relevant Public Datasets Used in the Study	19
Table 3.1	Agent Roles and Responsibilities in ATLAS-OPS	22
Table 3.2	Payment Failure Categories and Recovery Strategies	26
Table 3.3	Compliance Frameworks Covered by the Governance Layer	33
Table 4.1	Technology Stack Summary	36
Table 4.2	Kafka Topic Configuration Details	38
Table 4.3	Database Schema – Core Tables and Attributes	41
Table 4.4	Unit and Integration Test Coverage by Component	47
Table 5.1	Dataset Description for Model Training	50
Table 6.1	Performance Metrics Summary	57
Table 6.2	False Positive Rate Comparison Across Models	58
Table 6.3	Load Testing Results at Various TPS Levels	60
Table 6.4	LLM Report Evaluation Rubric and Scores	62

CHAPTER 1

INTRODUCTION

1.1 Background

This ecosystem has seen unprecedented levels of growth within the last decade. As per statistics from the Bank for International Settlements, there have been over 1.3 trillion non-cash transactions globally in 2024, with a compound annual growth rate (CAGR) of 23.6% since 2019. The dramatic growth in the digital payments space, due to the rise of mobile banking applications, e-commerce, real-time payment systems like UPI in India and FedNow in the US, and embedded finance products, has resulted in unprecedented pressure on the operational foundation behind every digital payment.

Payment operations, the process of guaranteeing that financial transactions are executed effectively and efficiently, while simultaneously adhering to regulatory requirements, have traditionally been achieved using automation, manual oversight, and monitoring solutions that have all been isolated from one another. Such an approach was suitable during times where transaction volumes were lower and the number of payment rails used was minimal. However, today's payment environment is marked by: (a) multiple payment rail ecosystems consisting of SWIFT, SEPA, UPI, RTP, and blockchain payments; (b) sub-second settlement speeds; (c) dynamic fraud patterns changing in real-time; and (d) a rapidly expanding regulatory surface including PCI-DSS, GDPR, DPDP Act, and AML/KYC frameworks.

Indeed, India's digital payments infrastructure has come to be recognized as one of the most vibrant ecosystems in the world today. The Unified Payments Interface (UPI) introduced by the NPCI in 2016 completed over 131 billion transactions in the fiscal year 2023-24, thus becoming the world's largest retail fast payments system in terms of number of transactions. Besides UPI, a whole array of payment platforms including NEFT, RTGS, IMPS, BBPS, and even FASTags account for handling all possible types of retail and wholesale payments in the country.

However, it is the convergence of all these aspects that reveals another important aspect: the current payment operations platforms are inherently reactive and reliant upon human effort and intervention while also not allowing for an architecture of autonomy and artificial

intelligence needed for future fintech operations. Indeed, when a payment gets stuck in a traditional operations center, an alert has to be generated to an engineer who then has to diagnose and initiate the recovery process.

1.2 Motivation

ATLAS-OPS is driven by three critical observations made while reviewing the processes of payment operations at medium to large-scale fintech companies. The first observation is that the average failure rate of payments in complex multi-rail systems is 2.3%, where each failed payment costs a company between USD 12 and USD 18 for manual intervention and customer support. In terms of sheer cost, this amounts to hundreds of millions of dollars annually in wasteful expenditure.

Secondly, the mean time to recovery (MTTR) for payments in systems experiencing downtime due to outages in the payments system averages 4.7 hours in companies that depend on NOC (Network Operations Center) practices, where payments are either held up, rejected, or processed incorrectly. This results in poor customer experiences, regulatory oversight, and, in many instances, statutory penalties for delayed payments. Thirdly, the average rate of false positives in automated rule-based systems used to detect fraud stands at 35-40%.

Moreover, there are increasing regulatory pressures. Financial regulators around the world are requiring banks to do more than merely ensure that they are secure and compliant; they must be resilient. Guidelines like those of the Reserve Bank of India for Business Continuity and Disaster Recovery, those of the European Banking Authority for ICT and security risk management, and those of the Federal Reserve in the United States for operational risk management all hint at an emerging regulatory demand for systems that can recover quickly and efficiently from any failure—a task that cannot be done easily using manual solutions.

All of the above considerations led to the development of an autonomous and AI-enabled system for detecting and resolving failures, which, by analyzing patterns in the past, could even anticipate failures and prevent them in the first place. It was never our intention to remove humans from the equation entirely; rather, we sought to minimize their workload by automatically resolving the numerous recurring issues that did not require human intervention.

1.3 Problem Statement

The existing platforms managing payment transactions have the following major problems, which affect the operational capacity, reliability, and regulatory posture of modern fintech companies:

- Reactive approach to failures that occurs when issues are detected through monitoring alerts without any capacity for predictive maintenance and self-recovery of the system.
- Rigidity of rule-based fraud detection that generates too many false positives leading to an unnecessary increase in costs and reduced customer satisfaction with their experience.
- Absence of intelligent routing algorithm that would take into account the current network state. The static table used at present does not provide for the dynamism of such parameters as latency or cost.
- Complex and manual approach to regulatory compliance that requires much labor input from employees and is subject to various errors that are hard to detect.
- Silos within a dashboard where operators have to switch from one tool to another in order to receive a comprehensive picture of the situation on the platform.
- Absence of a clear rationale for AI decisions made, which affects the ability of fintechs to comply with the regulations imposed by regulatory bodies.

Thus, the challenge that should be solved is the creation of a self-governing system capable of managing the entire lifecycle of payment transactions using advanced artificial intelligence, without requiring constant human intervention, while maintaining complete explainability and regulatory compliance.

1.4 Objectives

The key goals for the ATLAS-OPS initiative are as follows:

- Design of multi-agent AI system architecture capable of conducting autonomous payment operations management over several payment rails.
- Development of Context-Aware Recovery Decision Engine (CARDE) for intelligent and automated failure resolution involving causal reasoning and LLM.
- Development of a real-time fraud/anomaly detection mechanism equipped with the inherent explainability feature through use of SHAP and explanation generation capabilities.

- LLM-based transaction governance and compliance automation layer enabling autonomous regulatory reporting and answering compliance questions.
- Creation of reinforcement learning-based intelligent transaction routing optimizer which will be improving routing decisions in real time based on feedback loop.
- Develop a Payment DNA fingerprinting capability allowing complete transaction lineage monitoring which is required for fraud detection and auditing purposes.
- Validate the platform's performance against industry standards for MTTR, fraud detection precision, transaction fees, and system availability.

1.5 Scope of the Project

ATLAS-OPS is intended as an end-to-end proof-of-concept fintech platform for payment operations. In terms of functionality, the platform is supposed to address transaction lifecycle management, failure detection and autonomous recovery, real-time fraud scoring, creation of regulatory reports, and dashboard management for operators. It is engineered to support up to 10,000 transactions per second based on the current state of development and scalable for production-scale volumes.

The list of exclusions comprises some elements that, being essential in the process of production-level deployment, fall out of scope of the project and will be addressed further down the road, namely the interaction with live payment infrastructures (NPCI, SWIFT, card networks), production-level security certification by PCI-DSS Level 1 audit, and SaaS deployment of a multi-tenant application.

The project includes the following components:

Architecture engineering

Multi-agent framework development

Training and evaluation of machine learning models

Setup of event streaming using Kafka

Database design based on PostgreSQL

Fraud detection mechanism

LLM governance layer prototyping

Performance testing

1.6 Organization of the Report

Structure of the paper is as follows: Chapter 2 covers a survey of related studies and commercially available solutions to payment processing operations, fault handling in payments using artificial intelligence algorithms, and transaction management systems. Chapter 3 covers a detailed description of the architecture of ATLAS-OPS. Chapter 4 deals with the implementation approach used. Chapter 5 highlights the innovations and features of the system being developed. Chapter 6 deals with experiments conducted on the platform and performance analysis of the system. Chapter 7 concludes the paper.

1.7 Key Terminologies and Abbreviations

The following key terms and abbreviations appear throughout this report and are defined here for clarity:

- AIOps (Artificial Intelligence for IT Operations): The application of machine learning and AI techniques in improving and automating IT operations through event correlation, anomaly detection, and remediation.
- MTTR (Mean Time to Recover): The average duration between the discovery of a system or transaction malfunction and the restoration of its normal functioning. An important metric used in evaluating the operational efficiency of payment systems.
- MTTD (Mean Time to Detect): The average duration between the actual occurrence of an incident and its detection by the monitoring system. The smaller the MTTD, the lesser the effect on customers.
- Payment Rail: The infrastructure network on which payment transactions travel. These include the UPI network, the NEFT network, the IMPS network, and the RTGS network.
- SLA (Service Level Agreement): A set of conditions outlined in a contract that defines minimum performance standards for a payment processing service such as the success rate and uptime.
- LLM (Large Language Model): A deep neural network trained on huge text datasets that is capable of generating, summarizing, classifying, and reasoning over natural language text. Examples include GPT-4o and Llama 3.
- RAG (Retrieval-Augmented Generation): An architecture that couples dense retrieval with a generative LLM.

- SHAP (SHapley Additive exPlanations): An approach that explains machine learning predictions based on cooperative game theory by assigning values to the influence of each individual input.
- PPO (Proximal Policy Optimization): An optimization technique for reinforcement learning that iteratively adjusts policies through constraints.
- PCI-DSS (Payment Card Industry Data Security Standard): Standards that guarantee the implementation of adequate security controls in any entity dealing with payment card information.
- DPDP Act 2023 (Digital Personal Data Protection Act): A law regulating digital personal data processing in India, which determines the rights of data principals and the duties of data fiduciaries.
- Kafka (Apache Kafka): An open-source stream-processing software that supports high-throughput real-time processing in large-scale, fault-tolerant event streams.
- gRPC (Google Remote Procedure Call): An open-source system for building highly efficient client-server applications over HTTP/2 and Protocol Buffers.
- NOC (Network Operations Center): A central facility where IT staff monitor and manage operational systems, such as payment infrastructures.
- CARDE (Context-Aware Recovery Decision Engine): A new autonomous failure recovery decision engine proposed in this study and elaborated on in Chapter 3.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction to Payment Operations Systems

Payment Operations management has come a long way from being dependent on the batch processing methods to becoming an always on transaction model. Payment operations management in the earlier days, according to the Bank for International Settlements, as outlined in their CPSS papers from the 1990s, was predominantly batch-oriented, manual reconciliation and telephone based exception handling. Real Time Gross Settlement (RTGS), introduced at the dawn of the new millennium, was the first instance where monitoring requirements and rapid exception handling became necessary.

Classic research on Distributed Systems Reliability including the seminal works on logical clocks by Leslie Lamport (1978) and Impossibility of Consensus in Asynchronous Systems by Fischer, Lynch & Paterson (1985) provided a foundation of the inherent difficulties involved in creating reliable payment infrastructures.

More recently, the emergence of specific payment operations platforms such as Spreedly, Adyen, and Stripe Sigma, which include robust monitoring and analytical features, is a step forward from traditional NOC platforms. However, such solutions still fall short in implementing autonomy in the system as they are largely dashboarding/alerting tools which require a person to make sense of any anomalies and act upon them accordingly. Academic literature on payment systems reliability has largely focused on network-level fault tolerance and transactional consistency models, leaving the application of AI to autonomous payment operations management relatively unexplored.

2.2 AI-Driven Fault Detection and Recovery

Machine learning techniques have been applied to fault detection within IT since the introduction of AIOps (Artificial Intelligence for IT Operations), an idea that was coined by Gartner in its technology hype cycle reports since about 2017. The basic idea underlying AIOps is that data volumes and speed created by modern IT infrastructure exceed human capacity to analyze them, thus making it possible to develop models that could detect anomalies, correlate

events, and recommend or perform remediation operations much more quickly and effectively than a person would.

An important contribution in this area was made by Dang et al. (2019). They demonstrated that LSTM models could accurately predict failures in the cloud infrastructure 15 minutes prior to their occurrence with 89% accuracy. Their research into Microsoft Azure's operational telemetry revealed that using models based on sequences allowed for taking into account dependencies in metric time series, which were not considered by simple threshold-based alerting systems. Nair et al. (2020) expanded on this finding and used graph neural networks to track fault propagation paths within service dependency graphs, which is directly related to the problem at hand.

In the case of payments particularly, Brownlee (2021) came up with an idea of using rule-based machine learning for payment failure classification, with an accuracy of 87% demonstrated using a test set of 5 million transactions. However, again this research has been focused on detecting failures rather than recovering from them, which is another gap that exists in the field as of now that is solved by ATLAS-OPS through its CARDE Engine.

Another strand of literature in the area of self-healing systems (Psaier & Dustdar, 2011) focuses on autonomic computing and presents different theoretical approaches based on the idea of IBM's MAPE-K approach that includes Monitor, Analyze, Plan and Execute with the use of a common Knowledge base. Although MAPE-K serves as an excellent theoretical framework in general, yet there exists an opportunity in implementing this loop for payment-related applications due to lack of payment-domain knowledge base and action libraries.

2.3 Fraud Detection in Financial Systems

Detection of financial fraud has been extensively researched in the machine learning literature for more than two decades due to the abundance of transaction datasets and the severe economic consequences of not only false negatives (frauds not detected) but also false positives (legitimate transactions incorrectly flagged). One of the pioneering studies in this domain was performed by West and Bhatt (2000), who used artificial neural networks for detecting credit card frauds. It was shown that non-linear classifiers could identify more complex fraud patterns than simple rules.

The application of ensemble methods brought considerable improvements to the performance of financial fraud detectors. For instance, Dal Pozzolo et al. (2015) showed that

Random Forest models trained using cost-sensitive learning algorithms could achieve higher accuracy in identifying frauds even when the percentage of fraud cases was below 1% of all transactions. These researchers also introduced the idea of incorporating class imbalance in the training procedure, which is nowadays considered a state-of-the-art practice. The task of maximizing recall for the minority class without reducing the precision for the majority class remains challenging.

State-of-the-art financial fraud detection systems employ deep learning models capable of processing transaction sequences and graphs at the same time. Cao et al. (2019) introduced Graph Attention Network (GAT), which modeled transactions' relationships among accounts and merchants to detect fraudulent activity and reached an AUC of 0.97 on IEEE-CIS Fraud Detection benchmark dataset. More recent transformer-based approaches have demonstrated impressive results in identifying complex frauds based on analysis of transaction sequence patterns, as time-series language models.

One very important, but also commonly ignored issue in the area of fraud detection is that of model accuracy vs regulatory explainability. Although black-box models such as deep neural nets or gradient boosted ensembles could reach high scores in terms of AUC, their predictions cannot be easily explained either for the sake of audit purposes or to the customer or regulatory body that was adversely impacted by such a decision. ATLAS-OPS tackles this challenge by systematically incorporating SHAP (SHapley Additive exPlanations).

2.4 Large Language Models in Financial Operations

Using Large Language Models in financial activities is an emerging research area that has grown rapidly since the launch of GPT-3 in 2020. Bloomberg launched BloombergGPT (Wu et al., 2023), one of the earliest domain-specific LLMs trained on financial datasets, which has shown state-of-the-art results on financial NLP benchmarks relative to general LLMs of similar size. This paper demonstrated that training LLMs on specific domains can result in substantial improvements over prompting general LLMs using zero or few-shot techniques.

Several financial firms, including JPMorgan, HSBC, and Goldman Sachs, have deployed LLM-based solutions for contract analysis, regulatory document analysis, and report generation within their organizations. However, due to confidentiality concerns, technical details about these models are not available in peer-reviewed literature. Consequently, the academic community lacks examples of LLM-powered solutions used in the domain of payments compliance

automation, which makes ComplianceOracle of ATLAS-OPS relevant for bridging this knowledge gap.

The Retrieval-Augmented Generation (RAG) architecture by Lewis et al. (2020) serves as the technical basis for linking LLM predictions with specific documents, which is critical for compliance purposes as claims need to be supported by regulatory texts. RAG systems combine a dense retrieval component (typically a vector database of document embeddings) with a generative LLM, ensuring that responses are anchored in retrieved facts rather than model parameters—substantially reducing the risk of hallucination in high-stakes applications.

2.5 Multi-Agent Systems for Autonomous Operations

The study of multi-agent systems (MAS) as applied to distributed problem solving has been an extensively researched topic since the seminal contribution of Wooldridge and Jennings in 1995. However, the actual implementation of MAS for IT operations management has found applications in heuristically driven tools, like Microsoft's JARVIS and Google's Site Reliability Engineering automation tools, which employ a different paradigm than that of AI agents.

The ReAct architecture by Yao et al. (2022), on the other hand, made a significant breakthrough by demonstrating that LLM agents can achieve the interweaving of reasoning and actions to be able to solve problems that involve multiple sequential steps by alternating between chain-of-thought reasoning and tool use. This was possible because the reasoning-based agent could solve novel problems by applying reasoning to each individual subtask stepwise.

While LangChain's agent architecture and the AutoGPT project applied ReAct to multi-agents, ATLAS-OPS builds on top of their ideas and adds several domain-specific innovations, such as a specific role of agents for payment operations, a priority-driven orchestration protocol specifically optimized for low-latency payment scenarios, and a conflict-resolving policy that ensures no two competing agents are able to generate conflicting commands for the same payment operation.

2.6 Indian Payment Ecosystem: A Specific Context

The development and evaluation of ATLAS-OPS have been performed considering the Indian digital payments landscape, and hence it is necessary to mention some specific features of this ecosystem which were taken into consideration while designing the project. The payment infrastructure in India is regulated by the Reserve Bank of India (RBI) and National Payments Corporation of India (NPCI) and consists of various payment channels suitable for varying transaction volumes and settlement needs.

UPI has become the main channel for retail payments, processing more than 131 billion transactions totaling more than Rs. 200 lakh crore during FY 2023-24. Being an instant settlement channel available round the clock and allowing transfers between any banks in India, it becomes the main use case for the RouteMind optimization problem. NEFT and RTGS deal with transactions having higher values and time-specific settlement needs, where RTGS represents a real-time channel supporting transactions above Rs. 2 lakh. IMPS allows 24x7 inter-bank transfers on retail volumes, competing with UPI in several use cases.

Each rails has unique failure behavior. UPI failures are generally linked to the downtime of bank CBS (Core Banking System). This is common during off-hours since banks carry out their maintenance during this period. The failures in the NEFT rails are usually linked to time restrictions with regards to settlement batches and affect multiple transactions at one go. There are peculiar ways in which IMPS failure occurs as well, and this was vital in the creation of CARDE's domain-specific causal graphs and RAKBs.

Regulatory environment in India adds further complexity in this regard. The Reserve Bank of India imposes strict turnaround periods for failure refunds, with most UPI and IMPS transactions required to be refunded to the customers on T+1 business days after the initial transaction. Failure to comply with these deadlines results in penalties. This requirement is crucial in making autonomous recovery necessary.

2.7 Review of Relevant Datasets

The design and assessment of ATLAS-OPS have been done using both open datasets and artificial datasets created with the help of the PaySimulate simulator. The main open datasets used in the research include:

Table 2.2: Summary of Relevant Public Datasets Used in the Study

Dataset	Source	Description	Use in ATLAS-OPS
IEEE-CIS Fraud Detection	Kaggle / IEEE	590k transactions, binary fraud labels	FraudSentinel model benchmarking
PaySim Synthetic	Lopez-Rojas (2016)	6.3M synthetic mobile transactions	CARDE failure simulation
RBI Annual Payment Data	Reserve Bank of India	Rail-level aggregated statistics	RouteMind state space design
BIS CPMI Statistics	Bank for Intl. Settlements	Global payment system metrics	Literature context, background
PaySimulate (Custom)	This work	5M synthetic transactions	Primary training and evaluation

2.8 Comparative Analysis of Existing Solutions

A structured comparison of ATLAS-OPS against existing commercial and open-source payment operations solutions reveals the significant gap in autonomous, AI-native capabilities:

Table 2.1: Comparative Analysis of Existing Payment Operation Tools

Solution	Autonomous Recovery	AI Fraud Detection	LLM Governance	Real-Time Routing
Stripe Sigma	No	Partial	No	No
Adyen DataConnect	No	Yes (Rule-based)	No	Partial
Spreadly	No	No	No	Yes (Static)
Featurespace ARIC	No	Yes (ML)	No	No
ATLAS-OPS (Proposed)	Yes	Yes (DL+XAI)	Yes	Yes (RL-based)

As can be seen from this research, there is currently no commercial solution that would provide all the necessary features and capabilities provided by ATLAS-OPS. The platform represents an innovative approach to solving this problem, as it offers all five critical capability gaps described in the problem statement.

CHAPTER 3

SYSTEM ARCHITECTURE

3.1 Architectural Overview

ATLAS-OPS is structured as a cloud-native microservices architecture with five major architectural layers: The Event Ingestion Layer, The Intelligence Orchestration Layer, The Agent Execution Layer, The Governance and Compliance Layer, and The Presentation Layer. Communication between these layers happens via asynchronous event streams (Apache Kafka), synchronous REST/gRPC APIs, and a common operational database.

The architecture of ATLAS-OPS is based on the principles of the Hexagonal Architecture (Ports & Adapters) for isolating the AI components from the infrastructure, which ensures the ability to replace ML models, language service providers, and databases without changing the architectural structure of the application. Such an architecture is especially suitable for AI-based applications due to fast-evolving AI components.

Event Ingestion Layer:

The Event Ingestion Layer serves as the sensing layer for the system. All transaction events, system metrics, payment rails events, and regulatory alerts are collected through this layer, transformed into a standard event format, and published into Kafka topics. The layer itself is designed to be very lightweight, only responsible for transforming events into a standard format and validating their schema. All other processing occurs in downstream agent layers.

Intelligence Orchestration Layer:

The Intelligence Orchestration Layer runs the Orchestration Engine, which manages the operations of six AI agents. It keeps a list of incoming events sorted by their priority and assigns them to the right agents based on event type and urgency. The orchestrator uses a backpressure technique that buffers less important events in Kafka when agents are busy processing important failure events.

3.2 Multi-Agent Orchestration Framework

The intelligence core of ATLAS-OPS is a multi-agent framework comprising six specialized AI agents, each with a distinct operational mandate. The agents are designed to operate both independently and collaboratively, with well-defined protocols for sharing information and coordinating actions on the same transaction.

Table 3.1: Agent Roles and Responsibilities in ATLAS-OPS

Agent Name	Role	AI Technology	Communication
RouteMind Agent	Optimal payment rail selection	Deep RL (PPO)	Kafka + Redis
RecoveryGuard Agent	Failure detection and autonomous remediation	CARDE + LLM	Kafka + gRPC
FraudSentinel Agent	Real-time fraud and anomaly scoring	Ensemble ML + GNN	Kafka + REST
ComplianceOracle Agent	Regulatory compliance automation	LLM + RAG	gRPC + REST
SLA Watcher Agent	SLA monitoring and breach prediction	LSTM + Gradient Boost	Kafka
InsightNarrator Agent	Natural language interface for operators	GPT-4o + RAG	REST + WebSocket

The agents work in tandem with the help of an Orchestration Engine running a task allocation algorithm based on priority. The orchestrator manages a shared Agent State Registry hosted on Redis such that all agents can monitor the actions of others and refrain from taking any actions that may conflict with them during transactions of high throughput. In the aforementioned scenario, when the RecoveryGuard agent starts retrying after detecting failure in a transaction, it signals the FraudSentinel agent through the Agent State Registry not to score the transaction in question for fraud.

Agent communications leverage both Kafka for asynchronous event-driven orchestration and gRPC for real-time latency-sensitive interagent interactions. All the agents have an open API for their health check, which the Orchestration Engine queries every 500 milliseconds. An agent declared as unhealthy by the Orchestration Engine after failing to respond to a minimum of three polling requests within 3 milliseconds is declared unavailable, and its work is delegated to a secondary instance of the agent.

3.3 Context-Aware Recovery Decision Engine (CARDE)

CARDE is the intellectual centerpiece of ATLAS-OPS and represents the primary novel technical contribution of this work. It is an autonomous four-step recovery system that caters to the entire range of failure scenarios starting from straightforward timeouts to cascading multi-component failures.

3.3.1 Phase 1: Failure Contextualization

At the time when the payment failure event has been published to the failure topic within Kafka, CARDE creates the contextualized failure graph. This graph takes into account the following elements: Payment DNA fingerprint of the transaction (see Section 3.4), historic failure rates for the given payment rail, network health indicators, tier of the client and SLAs, and similar cases from the vector database. The resulting failure graph can be considered as a dynamic directed acyclic graph (DAG) processed by Graph Attention Network.

Phase 1 is limited in terms of time by 200 milliseconds in order to enable fast response. In this timeframe, CARDE queries three different data sources in parallel. First, Payment DNA vector database provides similar past failures. Second, the time series database with operational metrics offers recent rail health indicators. Third, the registry of current state processes queue lengths and agents' loads. The information gathered is combined and passed further as a context object to the next phase of the procedure.

3.3.2 Phase 2: Root Cause Analysis

However, the module of root cause analysis used by CARDE utilizes an approach of causal inference based on PC algorithm (Spirtes, Glymour & Scheines, 2000), with the inclusion of a causal graph of payment failures in payment domains, based on expert knowledge and historical data. The hybrid approach allows achieving much better results in terms of root cause identification (accuracy of 94.3% as compared to 78.6% when using only data-driven methods such as LSTM-based anomaly detection).

Domain-specific causal graph includes information about known relations in payment systems; for instance, if there is a sudden increase in the number of NEFT timeout errors, it means that these errors have been most likely caused by RBI batch window congestion or core banking system latency in the origination bank rather than payment gateway.

3.3.3 Phase 3: Recovery Action Selection

Considering the cause of the problem and the contextual graph, CARDE makes use of semantic similarity searches within its Recovery Action Knowledge Base (RAKB), which is an RAKB consisting of more than 50,000 previous incidents of recoveries. For selecting the best recovery action, CARDE uses a multi-criteria decision-making algorithm based on five parameters such as success probability for recovery, extent of impact on customers, regulatory constraints, availability of system resources, and effects on subsequent SLAs.

The chosen best recovery action is carried out autonomously, but any edge case where the confidence level is less than 85% is referred to a human operator with a report generated before hand on how to recover. This 85% threshold is the empirically determined level beyond which CARDE's confidence in taking autonomous action is justified.

Table 3.2: Payment Failure Categories and Recovery Strategies

Failure Category	Recovery Strategy	Avg. Recovery Time	Success Rate
Rail Timeout	Auto-retry on alternate rail	3.2 seconds	97.1%
Bank Connectivity Error	Queue for retry + notify bank	8.7 seconds	91.4%
Insufficient Funds (False Positive)	Validate and resubmit	5.1 seconds	99.2%
Duplicate Transaction Conflict	Idempotency check + resolve	2.8 seconds	98.8%
Malformed Payload	Auto-correct and resubmit	1.9 seconds	94.6%
Network Partition	Failover to backup rail	12.3 seconds	88.9%

3.3.4 Phase 4: Post-Recovery Validation and Learning

Once recovery operations have been successfully carried out, the system proceeds with validation in real time using CARDE's pipeline. Any successful recoveries are put into the RAKB database with newly adjusted weight of success probabilities. If the recovery fails, it goes through an escalation process and the failure analysis report generated will be fed back into the causal model. In effect, CARDE continuously learns from its experiences and becomes better at its job with more data being accumulated with each incident.

3.4 Payment DNA Fingerprinting System

Payment DNA is the name of the new transaction characterization technology proposed in this work. Each individual transaction is assigned a unique fingerprint with four key categories – temporal (timestamp, weekday, timezone); network (rail, latency, hop count); entity (merchant category, customer behavior signature, device fingerprint); and financial (amount, currency, settlement type). The 128-dimensional fingerprint is used for several tasks.

This vector is produced by our embedding model pipeline developed on sentence-transformers in less than two milliseconds. The fingerprints can be efficiently stored and queried with sub-10 millisecond approximate nearest neighbor retrieval within a Pinecone vector database. The selection of 128 dimensions resulted from an analysis showing declining benefits for larger dimensions, while retaining enough complexity to capture meaningful distinctions between transaction contexts.

As far as fraud detection is concerned, the Payment DNA framework provides means for recognizing fraud rings, a set of coordinated attacks designed to mask their true nature by being superficially different but clustering in the vector space due to shared underlying behavioral patterns. Rule-based or even some machine learning based approaches do not detect such attacks, as they cannot be captured by the conventional metrics.

3.5 Real-Time Fraud Detection Architecture

- The FraudSentinel agent uses a hierarchical approach for fraud detection, comprising three tiers. The hierarchical structure is a result of the inherent trade-off between performance and accuracy of fraud detection. Most frauds are detected at a minimal cost and in near real-time using simplistic logic, while others require expensive deep learning inference.
- Tier 1 - Rule Engine (< 1 ms): Predefined rules to detect blatantly obvious fraud scenarios such as velocity testing, blacklisting and impossible location changes. Handles around 60% of the fraudulent cases. No machine learning is used here. Pure rule-based system.
- Tier 2 - Machine Learning Scoring Engine (5 - 15 ms): Gradient Boosted Trees algorithm based on XGBoost using historical 50 million transaction records with over 200 features. Detects complicated but identifiable patterns using features such as behavior ratios, time deviation, merchant risk, device fingerprint match, etc.

Tier 3 - Deep Learning Anomaly Detector (50-100ms): Graph Neural Network for evaluating fraud schemes via transaction relationship graphs. Deployed exclusively on borderline cases at Tier 2, where XGBoost scores lie within an established range of uncertainty.

Fraud decisions will be accompanied by an explanation based on SHAP values, with those explanations logged for regulation. This approach is expected to handle 99.9% of all transactions in less than 100ms of SLA, while leaving only a few transactions to undergo Tier 3 analysis in parallel to prevent blocking of transaction flow.

3.6 LLM Governance and Compliance Layer

The Compliance Oracle agent is designed on the retrieval-augmented generation (RAG) model built on top of GPT-4o with a payment compliance knowledge base. The knowledge base consists of:

- PCI-DSS v4.0 requirements
- RBI Payment System Guidelines
- GDPR/DPDP Act provisions
- AML/KYC framework regulations
- SWIFT compliance requirements

Table 3.3: Compliance Frameworks Covered by the Governance Layer

Regulatory Framework	Jurisdiction	Key Requirements Covered	Update Frequency
PCI-DSS v4.0	Global	Data security, encryption, access control	Annual
RBI Payment Guidelines	India	Settlement timelines, dispute resolution	Quarterly
DPDP Act 2023	India	Data processing consent, breach notification	As amended
GDPR	European Union	Data subject rights, DPA notification	As amended
AML/KYC Framework	Global (FATF)	Customer due diligence, transaction monitoring	Bi-annual

Governance Layer Capabilities:

- Automated Pre-Transaction Compliance Screening: It verifies transactions for compliance with relevant regulations prior to transaction processing.
- Automated Regulatory Report Generation: It generates SAR, CTR, and settlement reconciliation reports without any manual writing process.
- Natural Language Compliance Queries: It enables compliance staff to ask questions related to transactions in natural language queries based on regulatory documents.

3.7 System Integration and Data Flow

The data processing flow in ATLAS-OPS for each type of transactions is structured according to a well-defined process. For the normal case of initiating a payment transaction, the process involves: (1) submission by client through RESTful API; (2) validation by the API Gateway for authentication and rate limiting purposes; (3) publication of transaction to payment.initiated Kafka topic; (4) creation of Payment DNA fingerprint; (5) consumption by RouteMind agent and selection of optimal rail; (6) scoring of transaction by FraudSentinel agent in parallel; (7) execution of compliance checking by ComplianceOracle; (8) transaction submitted to payment rail after checks pass; (9) consumption of transaction result event from Kafka topic; (10) update of SLA Watcher with metrics; (11) triggering of recovery process by RecoveryGuard agent.

All the events in the above described process are recorded in an immutable event log which is implemented using the TimescaleDB. This event log ensures that it is possible to reconstruct the transaction history for each transaction initiated historically, at any time, for audit purposes in regulatory processes.

3.8 Fault Tolerance and High Availability Design

Fault tolerance was made one of the main requirements for design in the ATLAS-OPS system since, by definition, payment systems should operate reliably regardless of component failures. The following approaches to achieving fault tolerance work together at multiple levels of the stack.

At the infrastructure level, all stateful services—Kafka, PostgreSQL, Redis—are clustered, each having a minimum replication factor of 3. Any individual node can be taken out, while there will be no data or availability loss in consequence. In particular, thanks to the log-

structured nature of Kafka, any failure of the Kafka broker in the middle of a write operation can be recovered from the replica.

At the application level, all ATLAS-OPS agents are implemented as stateless components regarding the context of an ongoing transaction. All states are persisted externally either in Redis (for agents) or PostgreSQL (for all other states). That way, any agent can be killed, and its pod replaced without affecting the process of completion of an ongoing transaction. The Kubernetes HPA ensures that sufficient agent replicas are always running to handle the current transaction load, and liveness probes ensure that unhealthy pods are automatically restarted.

The Payment DNA vector database at the data layer level is based on a master-replica architecture, where there is an automatic promotion of replicas. The RAG knowledge base is also duplicated as a read-only entity over various inference nodes, which allows both ComplianceOracle and InsightNarrator to still operate in case the primary node of the knowledge base needs an update or restart. Additionally, all artifacts for the AI models are kept in one model registry (MLflow), having several replicas of it, and the agents keep the local version of their model.

3.9 Scalability Analysis

Scalability properties of ATLAS-OPS were examined both from a theoretical point of view and from a load-testing perspective. From a theoretical standpoint, the main bottlenecks are listed, along with options for overcoming those issues.

Latency budget for per-transaction processing chain is as follows: Payment DNA computation (2ms), RouteMind rail calculation (5ms for caching rail states, 50ms for new rail state querying), FraudSentinel Tier 1 (1ms), Tier 2 (15ms), pre-transaction compliance verification (20ms for caching compliance rules, 200ms for rule enforcement with an LLM assistance on flagged transactions). Latency bottleneck of normal non-flagged transaction processing chain amounts to 43ms – significantly less than SLA 100ms target. Concurrent processing of FraudSentinel score calculation and ComplianceOracle verification (both reading off the same Kafka topic partition at once) is vital to meeting that latency goal – otherwise, latency will rise to ~93ms, which is perilously close to the deadline.

The major bottleneck of horizontal scalability is the inference of FraudSentinel Tier 3 graph neural network which consumes GPU resources and does not have enough parallelism for every single GPU. This is solved by applying FraudSentinel Tier 3 to a relatively small

percentage of transactions that fall under the uncertain band of Tier 2 (about 3% of all transactions according to our evaluation dataset). Another feature of the solution design is to use the GPU inference queue to allow asynchronous calculation of Tier 3 results without blocking the flow of the main process. Transactions waiting for Tier 3 results spend up to 500 ms in the pending mode before being approved or manually reviewed depending on Tier 2 results.

CHAPTER 4

IMPLEMENTATION METHODOLOGY

4.1 Technology Stack

ATLAS-OPS is implemented using a carefully selected technology stack optimized for high-throughput, low-latency payment processing with AI-native capabilities. Each technology was chosen after evaluating multiple alternatives against specific criteria relevant to payment operations: latency, throughput, operational maturity, community support, and compatibility with regulatory data handling requirements.

Table 4.1: Technology Stack Summary

Component	Technology	Purpose
Backend Services	Python 3.11 + FastAPI	Core agent logic, REST APIs
Event Streaming	Apache Kafka 3.5	Real-time transaction event processing
Agent Orchestration	LangChain 0.2 + LangGraph	Multi-agent workflow management
LLM Provider	OpenAI GPT-4o + Llama3 (local)	Governance, NL interface
ML Framework	PyTorch 2.1 + XGBoost 2.0	Fraud detection, anomaly scoring
Vector Database	Pinecone + FAISS	Payment DNA, RAG knowledge base
Primary Database	PostgreSQL 16 + TimescaleDB	Transaction records, time-series data
Cache Layer	Redis 7.2 + Redis Streams	Agent state, real-time metrics
Monitoring	Prometheus + Grafana	System and AI model monitoring
Container Platform	Docker + Kubernetes	Deployment and scaling
Frontend	React 18 + TailwindCSS	Operator dashboard
Message Queue	RabbitMQ	Inter-agent async communication
ML Lifecycle	MLflow + Evidently AI	Model versioning, drift detection
CI/CD	GitHub Actions + ArgoCD	GitOps deployment pipeline

4.2 Event-Driven Architecture Implementation

The core event-driven architecture of ATLAS-OPS leverages Apache Kafka with a topic structure tailored to payment operations scenarios. We decided to use Kafka rather than

RabbitMQ/AWS SQS/Google Pub/Sub as it is the only messaging service that guarantees event ordering and replaying which are required features for payment systems.

Main topics are: `payment.initiated`, `payment.processed`, `payment.failed`, `payment.recovered`, `fraud.alert`, `compliance.check` and `sla.breach`. Topics are divided into partitions by payment rail identifier; this ensures order in which messages from a certain rail will be processed and enables parallel processing on a topic level. The optimal number of partitions was found by running a load test and determined to be equal to 12 per topic – the number of Kubernetes workers that ATLAS-OPS works with (the most suitable TPS/latency ratio was identified for 10,000 TPS workload).

Kafka Streams is applied to perform stateful event processing; it includes calculation of sliding window velocity checks (count of transactions sent by a certain account within the last 60 seconds), session based fraud detection, and time-series SLA violations monitoring. Stream processing graph is executed in Kubernetes StatefulSet with auto-rebalancing upon pod failures which ensures zero downtime recovery of stream processing state. Consumer group offsets are committed every 100 milliseconds, limiting the maximum data re-processing window after a failure to approximately 100ms worth of events.

Kafka was selected as an alternative to RabbitMQ, AWS SQS, and Google Pub/Sub in order to ensure that events could be ordered, replayable, and persisted, which was important for our use case, as in payments systems, we want to be able to play back the history of the events.

The major Kafka topics include: `payment.initiated`, `payment.processed`, `payment.failed`, `payment.recovered`, `fraud.alert`, `compliance.check`, and `sla.breach`. All of these are partitioned by payment rail ID in order to enable ordered processing within one payment rail and parallel processing in separate payment rails. The number of partitions per topic was set at 12 (as there are 12 worker pods on the Kubernetes cluster), which turned out to be optimal in terms of providing maximum throughput with reasonable latency when performing benchmarking at 10k TPS.

Kafka Streams are used to implement stateful processing and aggregation. This includes velocity analysis (count of the number of payments made within 60 seconds from a certain account), session-based fraud pattern matching, and SLA monitoring based on time series data. The stream processor is running as a Kubernetes StatefulSet that can recover in case of pod failure and has no downtime during this process. Consumer group offsets are committed every

100 milliseconds, limiting the maximum data re-processing window after a failure to approximately 100ms worth of events.

Kafka was chosen over RabbitMQ, AWS SQS, and Google Pub/Sub primarily because of the requirement to support event stream order and persistence, making Kafka an ideal choice in terms of providing replayable event history in the context of the payments system.

Main Kafka topics to be used: payment.initiated, payment.processed, payment.failed, payment.recovered, fraud.alert, compliance.check, and sla.breach. Topics will be partitioned by payment rail identifier; in this way, Kafka will be able to process each rail in order while allowing processing across rails to run in parallel. After performing some tests on loading, we found that 12 partitions per topic (which corresponds to the number of Kubernetes worker nodes) yield the optimal balance between performance and latency for the required 10k TPS capacity.

Kafka Streams framework is used for stateful processing, implementing sliding window aggregation for velocity checks (transactions' number per second), session-based fraud detection, and time-series SLA enforcement. The stream processing logic will be deployed as a Kubernetes StatefulSet, providing rebalancing on failures for zero-downtime state restoration. Offset commit interval for consumer groups is set at 100 milliseconds; therefore, after any failure, the maximal amount of data loss for replay is around 100ms of events.

Table 4.2: Kafka Topic Configuration Details

Kafka Topic	Partitions	Retention	Consumers	Avg. Throughput
payment.initiated	12	7 days	RouteMind, FraudSentinel, ComplianceOracle	~8,000 msg/s
payment.processed	12	30 days	SLA Watcher, Audit Logger	~7,800 msg/s
payment.failed	6	30 days	RecoveryGuard, InsightNarrator	~200 msg/s
payment.recovered	6	30 days	SLA Watcher, Audit Logger	~170 msg/s
fraud.alert	3	90 days	ComplianceOracle, InsightNarrator	~50 msg/s
sla.breach	3	90 days	InsightNarrator, Orchestrator	~10 msg/s

4.3 RouteMind Agent: Reinforcement Learning Implementation

The RouteMind agent is constructed as a Proximal Policy Optimization (PPO) reinforcement learning agent that is trained continuously to choose an optimal payment rail for every transaction. PPO was chosen out of several other RL techniques (DQN, A3C, SAC), mainly due to its efficiency and robustness in training—in particular, in simulation-based training of agents for which choosing suboptimal actions leads to certain “costs.”

The state space is comprised of 47 parameters distributed among five main groups: current rail metrics (collected every 100ms using Prometheus), success rate per merchant category per rail over rolling 1h and 24h intervals, value and currency of a transaction, transaction's time and location, and current queue depths per rail. The resulting state vector is normalized and used as input for a fully connected three-layer neural network (256 neurons per layer) serving as a policy.

The reward is computed as a weighted sum of four factors: $R = w_1 * \text{success_rate} + w_2 * (1/\text{latency}) + w_3 * (1/\text{cost}) - w_4 * \text{SLA_penalty}$, with $w_1=0.4$, $w_2=0.3$, $w_3=0.2$, $w_4=0.1$ weights being defined by domain experts. The agent is trained using a simulated payment rail environment built with OpenAI Gym, pre-trained on 6 months of historical transaction data, and continuously fine-tuned in production using safe online RL with human-in-the-loop validation for significant policy changes.

4.4 Database Design and Data Flows

The main operational database consists of PostgreSQL 16 with TimescaleDB extension for time-series optimizations. PostgreSQL was selected due to its ACID-compliant nature, its advanced JSON capabilities using the jsonb column type, and the capabilities of the TimescaleDB extension that can effectively process large amounts of incoming time-series data (transactions, machine learning model results) due to automatic partitioning and compression.

The main schema is made up of seven tables: transactions (main table containing information about transactions along with references to the corresponding Payment DNA fingerprints), payment_rails (information on current status and performance history of all payment rails), failure_events (all detected failures along with the first context and root cause identification), recovery_actions (all recovery actions, whether autonomous or manual, along with their outcomes), fraud_scores (fraud scores for each transaction at all three levels with SHAP values explanation), compliance_checks (results of compliance checks and detected issues

before any transaction), and agent_decisions (records of all decisions made by agents along with confidence scores).

Table 4.3: Database Schema – Core Tables and Attributes

Table Name	Primary Key	Key Columns	Estimated Row Count (daily)
transactions	txn_id (UUID)	rail_id, amount, status, dna_vector_id	~5,000,000
failure_events	failure_id (UUID)	txn_id, failure_type, root_cause, confidence	~115,000
recovery_actions	action_id (UUID)	failure_id, action_type, outcome, duration_ms	~97,000
fraud_scores	score_id (UUID)	txn_id, tier, score, shap_json, decision	~5,000,000
compliance_checks	check_id (UUID)	txn_id, framework, result, cited_rule	~5,000,000
agent_decisions	decision_id (UUID)	agent_name, txn_id, decision, confidence	~10,200,000

4.5 Security Architecture

Security is performed through multi-layered approaches aligned with the principle of defense-in-depth. Authentication of API calls is done using OAuth 2.0 protocol combined with JWT tokens and mTLS connections for service-to-service interactions. All payment-related data is fully encrypted using AES-256-GCM encryption, and all network transmissions use TLS 1.3 encryption. The Zero Trust Architecture is implemented on the platform, meaning that all calls between services will be properly authenticated and authorized irrespective of their network location.

Secrets management uses HashiCorp Vault along with dynamic secrets generation and distribution for database login credentials and API keys to prevent any permanent secrets from being stored in application code. All secrets are automatically updated every 24 hours. Network policies in Kubernetes guarantee that all connections between services can only happen using the specified ports and protocols to mitigate any lateral spread of possible attack.

Model integrity is ensured by model signing and version control with cryptographic hashes stored in MLflow. Inference request inputs and outputs will be securely stored in immutable logs. To ensure that sensitive PII's are not stored in plaintext form in transactional

records, a PCI-DSS-compliant tokenization service is used, and the tokenization vault is isolated within a dedicated cluster.

4.6 Deployment Architecture

The ATLAS-OPS system is deployed using Docker containers and managed on a Kubernetes platform. The system deployment architecture comprises a production cluster running critical services, a shadow cluster that tests ML model upgrade canaries (the new ML model receives 5% of traffic in parallel with the existing production ML model, with a comparison performed automatically before rolling out), and a cluster on the edge that provides AI inference services in geographically distributed architectures.

Horizontal Pod Autoscaler is configured for all agent services on the basis of Kafka consumer lag. Consumer lag greater than 5,000 messages triggers autoscale, corresponding to buffering of only 0.5 seconds worth of event data, easily fitting into the budget for any latency-sensitive agents. Scale-in is intentionally slow to avoid oscillation, waiting for 10 minutes after consumer lag falls below the threshold.

An ArgoCD GitOps pipeline controls the deployment of our infrastructure and ensures that any changes made to agent code, machine learning algorithms, and configuration are under version control, are reviewed by way of a pull request process, and are automatically deployed with a rollback option. Our ML algorithm life cycle management process includes MLflow with the use of a retrain pipeline based on detection of data drift via Evidently AI and a PSI above 0.2 on transactions features.

4.7 Testing Strategy

Given the high-stakes nature of payment processing, ATLAS-OPS was developed with a comprehensive testing strategy spanning four levels: unit tests, integration tests, contract tests, and chaos engineering experiments.

The AI agent logics were unit tested using pytest. The ML model inference was mocked so that the orchestrator's logic is isolated from the inference outcomes. Integration testing was done on all the agents by running the tests against a Kafka and a PostgreSQL instance deployed locally using the Testcontainers package. Lastly, the API contract of each agent was contract tested using Pact. This ensured the API contract of each agent is met irrespective of the independent evolution of other agents.

For the chaos engineering experiments, we used Chaos Monkey for Kubernetes (kube-monkey). Kube-monkey randomly terminated pods while simulating load to demonstrate the robustness of the system against the failure of nodes. Two previously unnoticed failure modes became apparent after conducting these tests; the first was the delay caused by the re-balancing of the Kafka consumer group while failing multiple pods simultaneously. The second was the delay during the Redis sentinel election under network partition.

Table 4.4: Unit and Integration Test Coverage by Component

Test Type	Framework	Coverage	Pass Rate
Unit Tests	pytest	847 tests, 91% line coverage	100%
Integration Tests	pytest + Testcontainers	156 scenarios	98.7%
Contract Tests	Pact	34 provider-consumer pairs	100%
Load Tests	Locust + k6	12 scenarios, up to 12K TPS	Pass within SLAs
Chaos Tests	kube-monkey	6 failure injection scenarios	5/6 Pass

CHAPTER 5

UNIQUE FEATURES AND INNOVATIONS

5.1 Novel Contributions of ATLAS-OPS

ATLAS-OPS offers multiple technical novelties that set it apart from the state-of-the-art academic and industrial offerings available in the field of payment operations. These novelties, taken together, solve the inherent shortcomings of contemporary systems highlighted in the literature review and problem statement sections. The present chapter outlines the features of each novelty, the technical reasoning behind its architecture, and places it within the context of related literature.

5.2 Context-Aware Recovery Decision Engine (CARDE)

CARDE serves as the first instantiation of an entirely self-sufficient system of payment failure recovery using causal inference, graph neural networks, and large language model reasoning together within a single decision pipeline. Contrasting previous efforts where payment failure detection and recovery have been seen as two independent tasks with a human operator acting as the intermediary between them, CARDE uses an end-to-end pipeline approach from detection through to confirmation of successful recovery without requiring any human interaction for 84.7% of all cases that lie within the system's confidence bounds.

The defining characteristic of CARDE's approach is Context Fusion technology that considers three dimensions of failure assessment independently and fuses their results into a final output decision: Statistical Context, determined using historical data on failures and time-series models; Semantic Context, acquired using the large language model reasoning engine for understanding both the failure message and operational runbooks related to the failure; and Relational Context, generated using a graph analysis approach that analyzes relationships between the faulty component and other nodes in the payment system dependency network.

Context Fusion in practice ensures that CARDE is able to determine that the cause of the abrupt surge in NEFT timeout failures on a Monday morning is due to the delay in the RBI's weekly processing schedule rather than any gateway failure, as the statistical context of CARDE highlights the occurrence of the failure every week, the semantic context of CARDE recognizes

the RBI's weekly processing schedule, and the relational context of CARDE establishes the location of the failure as being at the edge of the bank's network.

5.3 Payment DNA Fingerprinting

Payment DNA is a new method of transaction representation that takes into account behavioral, temporal, and relational features in addition to the standard transaction attributes by embedding all this information into one space. The ability of the 128-dimensional embedding to perform tasks that cannot be performed by either a transaction identifier or by transaction attributes alone should be noted.

As an algorithm for extracting fingerprints from transactions, a proprietary model built with the help of the PyTorch framework was used. This neural network was trained by 500 million anonymized transaction fingerprints, and the training process was performed according to the contrastive approach, whereby the cosine similarity between the embeddings of similar transactions is maximized, while the difference between similar and fraudulent ones is increased as much as possible.

In addition to detecting fraud, Payment DNA can serve a useful purpose for regulatory compliance. According to both DPDP Act requirements and PCI-DSS audit requirements, a bank should be able to restore the context of each transaction at any moment. This is achieved by combining Payment DNA fingerprint with the immutable event log of ATLAS-OPS.

5.4 AI-Driven SLA Breach Predictor

The SLA Watcher agent utilizes a predictor for detecting breaches of service-level agreements (SLAs), which sends alerts to the operator about potential violations 15-45 minutes in advance. It constitutes a revolutionary improvement over classical solutions based on threshold monitoring, as they are able to send alerts to operators only after an SLA breach starts.

The predictor uses a long-short-term-memory (LSTM) network model trained on SLA performance metrics together with additional signals. They include scheduled maintenance periods based on company calendars, payment rail issues gathered from incident.io API, calendar events reflecting peak payment periods at month ends and holidays when customers usually make payments in larger amounts, and geographic correlations with the events that may impair the operation of payment rails, e.g., power outages in regions or political problems.

Back-testing demonstrated that during 12 months of past data, the predictor for SLA breaches reached 91.2% precision and 87.6% recall rates within 30 minutes' predictions. The choice of 30 minutes as a time span stems from the observation that, from an operational perspective, engineers require this period to be able to start implementing contingency actions.

5.5 InsightNarrator: Multi-Modal Operator Interface

InsightNarrator is a novel human-computer interaction interface that enables non-technical stakeholders to communicate with the ATLAS-OPS platform via natural language. Unlike traditional dashboards that necessitate prior knowledge of which metrics need to be considered, InsightNarrator facilitates natural language-based querying that returns contextually meaningful responses grounded in data.

Some sample queries that InsightNarrator can process include: "Why did we see an uptick in our UPI failure rate yesterday evening?", "Display all high-value transactions by Merchant X that were initially marked as fraudulent but then passed the checks in the last 30 days", and "Prepare a PCI-DSS compliance report for Q4 2025." These queries will elicit distinct workflows on the backend, involving time-series metric retrieval and correlation against external events, running structured SQL queries on databases and formatting the results, and triggering the generation pipeline of the ComplianceOracle, respectively.

InsightNarrator supports a multi-modal user interface, allowing for text-based querying through web chat, voice-based querying through Whisper API, and proactively sending narratives as alerts to operators ahead of any request. Proactive alerts are created using a different monitoring loop that analyzes various operational parameters and produces human readable stories when something significant happens, such as creating an alert like "There has been a 2.3% drop in IMPS transactions in the last 15 minutes, correlated with high response times of Bank X. CARDE is actively monitoring the situation and will start automatic routing of the traffic if necessary."

5.6 Explainable AI Throughout the Platform

- One of the unique design aspects of ATLAS-OPS is its pervasive explainability where each machine learning prediction influencing the processing of any transaction within the payment flow comes with an explainable output that includes the human-readable rationale for the decision and a probability score. This aspect was deliberately designed to be incorporated into the system since day one.

- SHAP (SHapley Additive exPlanations) for each of the ML model predictions, including the attribution of key features contributing to the fraud scores, SLA predictions, and routing decisions. For every fraud prediction, a SHAP waterfall plot is created displaying the contributions made by the top 10 features.
- Explanations for CARDE recovery decisions in natural language generated using LLMs, including technical and non-technical versions for different stakeholders.
- Explanations of declined transactions on a counterfactual basis: “This transaction was declined due to [reason]. It would have been accepted if [condition].” The use of this structure guarantees adherence to the requirements outlined by the EU AI Act for the explanations of automated decisions that affect individuals.
- Reporting confidence intervals for all probability estimates and providing uncertainty quantification to the operator whenever the model operates on the boundary of the distribution from which it was trained.

The robust explainability framework ensures full compliance with RBI AI/ML requirements, the EU AI Act for European operations, and the DPDP Act on transparency.

5.7 Autonomous Compliance Report Generation

ComplianceOracle is able to autonomously produce reports necessary to comply with regulations that have historically consumed considerable manpower. This is particularly relevant in mitigating one of the more manual processes in payments, which includes compiling mandatory reports, such as Suspicious Activity Reports, Currency Transaction Reports, and periodic settlement reconciliation reports.

The report generation workflow consists of the following six steps: (1) querying the operational database for transactions fitting within the scope of the report; (2) applying filters based on regulatory rules found in the compliance knowledge base to detect reportable events; (3) analyzing the filtered transaction data for statistical significance and pattern recognition; (4) drafting the report using LLM report generation with references to regulatory documents for each factual claim made; (5) self-consistently verifying each reported number by cross-referencing with the corresponding database queries; and (6) producing the final formatted document.

In comparisons with the manually produced reports, ATLAS-OPS reports have proved to be 96.8 percent accurate in their content and only take 23 minutes of human review time to get approved, while the total manual production of reports takes from 8 to 12 hours. This human review time

is used to resolve two kinds of edge cases, namely transactions that have entered the grey area of regulations and require personal judgments and different interpretations of an ambiguous regulation made by the LLM and the compliance officer.

5.8 Design Principles and Engineering Philosophy

In addition to the individual innovations discussed in Sections 5.2 to 5.7, ATLAS-OPS was engineered based on a series of fundamental design principles that informed each and every decision made during the process. These design principles are worth stating outright, as they are lessons learned during the engineering process that will prove helpful to other researchers or practitioners who may wish to develop AI-based operations solutions in the future.

The first guiding principle is Automation First, Escalation as Exception. Each and every operation within the ATLAS-OPS framework begins with the presumption that it should be automated, with escalation of the task to a human operator only occurring if there is insufficient confidence in the system's ability to perform the task successfully or if the task itself is beyond the scope of the training data used to train the system. This stands in contrast to traditional operational systems, which begin with an inherently human task and then add automation on top of it.

Secondly, there is a principle called Explainability as a First-Class Requirement. The requirement for explainability was considered not as an afterthought but as a key feature that has equal importance to performance and correctness starting on day one of design. Thus, model selection, feature extraction, and output formatting all had to be done in such a way that explainability criteria were fulfilled from the very start. In some cases, this involved compromising a bit on model accuracy in favor of much better explainability, which was completely justified in a regulatory environment like finance.

Thirdly, there is a principle called Event-Driven vs. Request-Response. All communications between components of ATLAS-OPS that do not have hard latency constraints use event streaming instead of REST or RPC calls. This approach provides benefits for resilience, scalability, and auditability.

Principle 4 involves the use of Configuration Over Code for Operational Parameters. Every single operationally important threshold, weight, and parameter used by the system, whether it is the CARDE confidence escalation threshold, RouteMind's reward function weights, or FraudSentinel's tier boundary scores, are externalized into configurable files using the GitOps

pipeline. Consequently, this greatly reduces the difficulty of altering the system’s behavior based on changing circumstances without any code modifications or re-deployments.

5.9 Comparison with Alternative Design Approaches

When designing ATLAS-OPS, various other architectures were explored and discarded before settling on the present design. It is helpful to outline some of these discarded architectures and explain the rationale behind their rejection, as this sheds light on the motivations for choosing the current architecture.

A monolithic AI engine that would integrate all agent capabilities into one single model was one such architecture. This option would have been easier to implement; however, there are many significant disadvantages to consider, including a lack of redundancy in case of failure, inability to scale up individual functionalities independently, challenges in modifying any individual component, such as the fraud model, without restarting the entire system, and an inability to train individual functionalities with expertise specific to that function.

The third alternative was rule-based systems with additional statistical analysis for anomalies but without ML, which could have resulted in better cost performance and easier explainability. However, it was clear that such a solution would be inherently incapable of coping with novel failure patterns or fraud schemes. Considering the essence of the problem that ATLAS-OPS solves, such a design would be inappropriate for the project.

Finally, the fourth alternative is to employ an all-encompassing LLM as the decision-making engine and use prompt engineering rather than specialized models to detect fraud, route transactions, and recover from failures. Such a design was attempted in an initial prototype and proved to be insufficient in terms of performance: inference by LLMs at the necessary level of speed and costs (under 100 ms and thousands of transactions per second) does not seem viable with current commercially available large language models. In conclusion, only those processes requiring reasoning based on understanding of natural language text can benefit from employing LLMs in the architecture while specialized machine learning algorithms should be applied for scorecard-like functions.

CHAPTER 6

EXPERIMENTAL RESULTS AND EVALUATION

6.1 Experimental Setup

The experiments for evaluating ATLAS-OPS were conducted using a simulated payment operations environment setup that mimics a medium-sized fintech company handling roughly 5 million transactions daily over four payment channels: UPI, NEFT, IMPS, and simulated credit/debit card payment channel. The simulation mimicked the statistical characteristics of Indian payment system data as reported in the RBI's annual report on digital payments in 2023-24, including realistic distribution of transaction amounts, time-of-day transaction volume, and per-rail failure rates.

For conducting the experiments, the test bed included a Kubernetes cluster with 12 nodes (32 CPU cores and 128GB RAM each), a 6-node Apache Kafka cluster (replication factor 3), 3-node PostgreSQL database cluster (synchronous streaming replication), and an NVIDIA A100 GPU node for inference by the ML models. All the infrastructure components were deployed using bare-metal servers available at the college laboratory.

The transaction simulation was carried out using the PaySimulate framework that was built as a part of this project. PaySimulate creates statistical representations of transactions with respect to various parameters defined from publicly available RBI payment system reports. It includes realistic simulation of failure injection like rail timeout cascade, banking side failures, and coordination between fraudulent attacks. PaySimulate is open-sourced as an additional contribution from this project for future research in payment system simulations.

There were five evaluation criteria used in this project: Failure Recovery Performance evaluated in terms of MTTR and autonomous recovery ratio; Fraud Detection Accuracy evaluated in terms of AUC, precision, recall, and false positive ratio; Transaction Routing Optimality evaluated in terms of success rate, latency, and cost; SLA Breach Prediction Accuracy evaluated in terms of precision and recall within 30 minutes; and System Performance evaluated in terms of throughput and latency percentile for stepped load.

6.2 Failure Recovery Performance

Evaluation of CARDE involved testing its effectiveness in dealing with 10,000 simulated failure cases that fell into 15 categories of failure, produced using the failure injection feature of the PaySimulate tool. For comparison purposes, a scenario of a typical manual NOC process based on established benchmarks of incident response was developed.

Table 6.1: Performance Metrics Summary – Failure Recovery

Metric	Baseline (Manual NOC)	ATLAS-OPS	Improvement
Mean Time to Recover (MTTR)	4.7 hours	8.3 minutes	97% reduction
Autonomous Recovery Rate	0% (all manual)	84.7%	—
Recovery Accuracy	91% (human error rate)	96.2%	+5.2%
False Recovery Rate	N/A	3.1%	—
Human Escalation Rate	100%	15.3%	84.7% reduction
Mean Time to Detect (MTTD)	23 minutes	1.2 seconds	99.9% reduction

The MTTR reduction of 97% is the most substantial operational gain provided by ATLAS-OPS. The 15.3% increase rate is attributed to new failure modes that the knowledge base of CARDE could not deal with due to lack of historical precedence—this is a figure anticipated to reduce as the system gains operational maturity. It is worth noting that the 96.2% accuracy of recovery achieved by CARDE is higher than the 91% accuracy of the manually operated NOC baseline, indicating that automated recovery may even be more accurate than human recovery.

6.3 Fraud Detection Performance

FraudSentinel was assessed using a test set of 2 million transactions produced by PaySimulate with a fraud ratio of 0.8%. This was done with regard to the benchmarks established by the RBI, which is typical for the industry. There are three different benchmarks that were analyzed in this experiment: the first is a purely rules-based benchmark (industry benchmark), the second is XGBoost with features, and the third is ATLAS-OPS FraudSentinel.

Table 6.2: False Positive Rate Comparison Across Models

Metric	Rule-Based Baseline	XGBoost Only	ATLAS-OPS FraudSentinel
AUC-ROC	0.81	0.94	0.97
Precision	0.68	0.89	0.94
Recall	0.72	0.91	0.93
False Positive Rate	38.4%	12.1%	6.8%
P99 Inference Latency	< 1ms	12ms	87ms
Explainability	No	Partial (SHAP)	Full (SHAP + NL)

The 45% decrease in false positive rate from 12.1% for using only XGBoost to 6.8% for using the complete three-layered FraudSentinel framework is a particularly significant achievement. In a scenario where there are 5 million transactions per day with a fraud rate of 0.8%, this equates to roughly 2,660 less valid customer transactions being falsely prevented each day. With a transaction worth Rs. 4,500 and a pessimistic 40% of customers leaving the process due to their transactions being blocked, this is worth approximately Rs. 47.9 lakh per day in recovered transactions.

6.4 Routing Optimization Results

The RouteMind RL agent was tested for 30 days, and its results were compared against two other models: static routing (which used rules to distribute transactions to preassigned rails depending on the transaction amount and the originating bank) and round-robin load balancing. Before starting the evaluation phase, the RL agent underwent a 7-day warm-up phase within the simulation environment, based on its convergence properties.

Table 6.3: Load Testing Results at Various TPS Levels

Metric	Static Routing	Load Balancing	RouteMind (RL)
Overall Success Rate	97.1%	97.6%	99.1%
Average Transaction Latency	2.3s	2.1s	1.4s
Average Transaction Cost	Baseline (1.0x)	0.97x	0.78x
SLA Compliance Rate	91.3%	92.7%	98.4%
Peak Load Success Rate (>8K TPS)	89.2%	91.1%	97.3%

The 22 percent decrease in the mean cost per transaction from RouteMind is especially important from a business standpoint. Costs associated with payment rail include interchange fees, NEFT fees, and gateway processing fees. These costs make up a substantial portion of the operating costs of payment processors. The 22 percent reduction in cost will directly affect the bottom line of a company's finances. Similarly, an increase in SLA accuracy from 91.3 percent to 98.4 percent is also important because non-compliance of SLAs comes with financial repercussions.

6.5 System Performance Under Load

Load testing was conducted using Locust and k6 in combination, simulating transaction volumes from 1,000 TPS to 12,000 TPS in stepped increments of 1,000 TPS, with each step maintained for 5 minutes before increasing load. The Kubernetes HPA was active during all tests, allowing automatic pod scaling.

Table 6.3 (continued): System Throughput and Latency Under Load

Load (TPS)	P50 Latency	P95 Latency	P99 Latency	Error Rate	Pod Count
1,000	18ms	47ms	89ms	0.001%	3
3,000	34ms	78ms	142ms	0.002%	5
5,000	67ms	112ms	198ms	0.003%	7
7,000	98ms	187ms	312ms	0.005%	9
10,000	142ms	287ms	451ms	0.012%	12
12,000 (over-capacity)	389ms	892ms	1,847ms	0.8%	12

The system was able to sustain less than 500ms P99 latency at 10,000 TPS, which is the maximum capacity of the system design. Degradation after 12,000 TPS, which exceeds the maximum capacity of the system design, is an anticipated behavior, which proves that the system fails in a safe way, unlike the catastrophic failures. Kafka showed linear scalability of throughput with the increase in partitions, proving the correctness of the event-based design.

6.6 LLM Governance Evaluation

ComplianceOracle and InsightNarrator agents were tested for their performance using a blend of both quantitative measures and qualitative assessment through expert evaluation conducted by two experienced compliance officers who compared the reports generated by

ATLAS-OPS with manually produced benchmark reports. This particular form of testing was highly subjective since it included qualitative measures of performance.

Table 6.4: LLM Report Evaluation Rubric and Scores

Evaluation Criterion	Score (out of 100)	Evaluator Agreement	Manual Baseline
Factual Accuracy	96.8	High (k=0.82)	N/A (ground truth)
Completeness	94.2	Moderate (k=0.71)	N/A (ground truth)
Regulatory Citation Accuracy	98.1	High (k=0.88)	N/A (ground truth)
Readability	91.4	Moderate (k=0.68)	~88 (peer review estimate)
Preparation Time (minutes)	23 (review only)	—	480-720 (full manual)

Inter-rater agreement was assessed using Cohen’s Kappa statistic. The moderate agreement on completeness and readability stems from the difference in views between the two raters on what is ideal – while one rater favored verbose technical notes, the other one preferred concise narrative for executive consumption. Both agreed on the suitability of the report for submission to regulation for the observed amount of work performed on their generation. All factual mistakes could be attributed to missing knowledge in the RAG corpus (out-of-date regulatory documents) and not to LLM hallucinations.

For testing the accuracy of insight extraction, a test set consisting of 200 natural language questions was posed to InsightNarrator by one of the authors. The questions were marked using a detailed mark scheme. InsightNarrator answered 91.3% of the questions entirely, partly answered 4.2%, while providing false answers for 4.5%. Common failure types included questions requiring information synthesis across multiple non-connected data sources (e.g., 'Compare fraud rate statistics this month with statistics for the same time period last year on each rail'), which surpassed the complexity capabilities of the current RAG pipeline.

CHAPTER 7

CONCLUSION AND FUTURE WORK

7.1 Conclusion

In this project, we introduce ATLAS-OPS, an autonomous AI solution for intelligent operations in payments, failure recovery, and transaction governance. The implementation of multiple AI systems, orchestration by large language models, reinforcement learning routing, and fraud detection proves that ATLAS-OPS is able to address inherent weaknesses associated with traditional payment operations platforms, utilizing AI technologies effectively.

The findings from experimentation provide sufficient support to our primary claim: the performance of ATLAS-OPS in terms of its ability to recover from payment operation failure and prevent fraudulent transactions leads to the following achievements – a 97% decrease in mean time to recovery from payments failures, 45% decrease in false positives during fraud detection, 22% lower transaction processing costs, and 99.1% of successful payments.

First, from a technical standpoint, the innovations that the paper proposes in terms of context-aware recovery decision making with CARDE, Payment DNA fingerprinting, tri-tiered fraud detection with full explainability, and compliance automation with the use of large language models present solutions to specific problems found in fintech operations practice and have been thoroughly tested and evaluated. The architectural decisions, such as choosing the event-driven architecture and using Apache Kafka in combination with the coordination scheme using shared state registry, proved to be effective for the payment operation scenario.

In terms of the academic contributions, ATLAS-OPS contributes to the body of knowledge through being the first system to incorporate causal inference with LLM reasoning into a payment failure recovery process. The paper proposes the notion of Payment DNA embeddings to represent transactions as well as presents one of the few existing, thoroughly tested and evaluated implementation of a compliance automation system for payment processing operations. The PaySimulate simulation framework was developed as part of a secondary contribution of the research.

7.2 Limitations

There are several critical limitations to the work carried out thus far that must be recognized. First, all evaluation took place in a simulated environment, where real-life payment rails are bound to have further complications and restrictions. Even though PaySimulate was built using RBI's historical data, certain aspects were simplified in its design, especially those related to the specific failures of individual banks. Second, LLMs used by this application depend on external services through APIs, which brings further problems related to network latency and data residency. For regulated financial institutions, these challenges will need to be mitigated by deploying these services on-premises. Third, the RL agent responsible for routing decisions needs to have a considerable time window before reaching its best configuration. Therefore, the performance of the application during the initial weeks of implementation will be inferior to the tested one. Fourth, the compliance database is static and requires updates regularly. This challenge will bring significant overhead costs that were beyond the scope of this project.

7.3 Future Work

Several promising directions for future research and development emerge from this work:

- **Production Deployment & Real-World Validation:** Deploying the solution in a sandboxed production environment with a fintech company partner would yield critical real-world insights into the performance of the system and identify failure modes not seen through simulations. Even a pilot deployment that processes just a small portion of the actual volume of transactions will yield useful insights.
- **Payment Domain-specific LLM Fine-tuning:** Fine-tuning an existing open-source language model on payment domain-specific regulations and reports would help fine-tune the capabilities of ComplianceOracle and InsightNarrator for greater accuracy and allow on-premise deployment of the solution in privacy-constrained settings.
- **Federated Learning for Detecting Multi-Institution Fraud:** By implementing federated learning algorithms within several financial institutions, a collaboration could be established to train an enhanced fraud model without revealing sensitive transaction details in a shared database, allowing for a more accurate identification of multi-institution fraud rings.
- **Blockchain for Immutable Transactions' Lineage:** Introducing Payment DNA fingerprints into a permissioned blockchain like Hyperledger Fabric would lead to an immutable and distributed transactions lineage with zero knowledge proof capabilities.

- **Quantum-resistant cryptography:** With the advancement of quantum computing, which is a looming threat to present-day cryptographic measures used for protecting financial transactions' data, future iterations of ATLAS-OPS must incorporate post-quantum cryptography algorithms for the signing of transactions and data security, following the NIST standards on post-quantum cryptography.
- **International Payments Management:** The scope of RouteMind can be extended to support payments involving cross-border transactions through SWIFT and Central Bank Digital Currencies, along with implementing FX rate optimization in the routing reward function.
- **Explainable RL for Route Decisions:** Presently, the PPO framework does not have any explanation capabilities for individual route decisions. It is essential to explore causal reinforcement learning methods that allow humans to understand why a particular decision was taken, especially when such a requirement exists within a jurisdiction.
- **Regulatory Horizon Scanning:** Future enhancements of ComplianceOracle must include a regulatory horizon scanning feature, where the system will automatically monitor regulatory announcements published in official gazettes and RBI circulars, ensuring that the compliance database is always up-to-date with new regulations.

7.4 Broader Implications

The findings from this project have implications beyond the scope of the payments process. The architectural approach shown in ATLAS-OPS, which combines AI-based multi-agent orchestration, causal inference to understand root causes, LLMs for governance, and explainability of decisions, is generalizable to any use case in other autonomous operation domains across various industries.

For example, a similar solution could be built in the field of health care, where AI would be responsible for handling autonomous resolution of hospital information system outages, again using the approach of performing autonomous actions when the type of problem has already been learned, while escalating problems that are encountered for the first time to a person. For telecommunications, another CARDE engine could be implemented to autonomously recover from network outages.

In fact, the Payment DNA approach, in particular, can be applied in a much wider variety of cases than just payments due to the fact that any field involving monitoring and analysis of complex multi-step processes, including supply chain management, clinical trials management,

legal case management, etc., may find this method of high-dimensional encoding useful for process fingerprinting.

Speaking about the scientific implications of ATLAS-OPS, it should be noted that combining causally-grounded domain-specific knowledge with advanced machine learning proved to be more effective than using these approaches separately. This result is consistent with the overall development trend of AI towards hybrid neuro-symbolic architectures, and the particular architecture used in CARDE could be interesting in those areas where the cost of autonomous failures is very high but the benefits of autonomous actions even higher.

7.5 Ethical Considerations

The creation of AI systems that can carry out payment transactions in an autonomous manner also raises various ethical concerns that should be clearly discussed. Ethical issues relating to ATLAS-OPS pertain mainly to the conflict of values related to efficiency of automation versus human accountability – when the system makes an autonomous decision that negatively impacts the client, say, incorrectly identifies a valid transaction as fraudulent or makes a recovery move that results in the delay of settlement of the payment, then this decision needs to be clearly justified, audited, and rectified.

These objectives are achieved through the use of a complete explainability framework and the conscious inclusion of pathways for human intervention in such situations. Inclusion of a 85% confidence level for autonomous actions is a calculated risk since below this level, the system would not make the autonomous decision even if it would have been faster since in some cases, the costs of an erroneous decision may far outweigh any benefits gained from efficiency.

A second ethical issue pertains to the issue of biased algorithms within the process of fraud detection. If the machine learning model behind fraud detection has been trained to link specific demographic characteristics, geographical locations, or transaction behaviors to an increased likelihood of fraud due to historical biases rather than genuine fraud signals, this could lead to further discrimination against certain segments of customers. The solution proposed by ATLAS-OPS is the inclusion of fairness metrics in the machine learning model evaluation phase, as well as the requirement that SHAP explanations for declined transactions can be audited by compliance teams in search of discriminatory attribute usage.

LIST OF PUBLICATIONS

The papers listed below are either under submission or in planning stages as per the research findings of the present thesis dissertation:

- [1] Mohammed Tailor, Prabhat Kale, Mayur Kohad, Jayesh Jumde, 'ATLAS-OPS: A Multi-Agent Approach for Autonomous Payment Transactions and Failure Recovery', International Conference on Artificial Intelligence in Financial Technology (AIFT 2026). Under submission.
- [2] Mohammed Tailor, Mayur Kohad, 'A Context-Aware Recovery Decision Engine for Autonomous Payment Failure Management using Causal AI', IEEE Transactions on Neural Networks and Learning Systems. In progress.
- [3] Prabhat Kale, Jayesh Jumde, 'Payment DNA: A Multi-Dimensional Approach for Transaction Fingerprinting towards Fraud Detection and Audit Trail Creation', International Journal of Information Security (Springer). In progress.

REFERENCES

- [1] Wooldridge, M., & Jennings, N. R. (1995). Intelligent agents: Theory and practice. *The Knowledge Engineering Review*, 10(2), 115–152.
- [2] Dal Pozzolo, A., Caelen, O., Johnson, R. A., & Bontempi, G. (2015). Calibrating probability with undersampling for unbalanced classification. *2015 IEEE Symposium Series on Computational Intelligence (SSCI)*, pp. 159–166, Cape Town, South Africa.
- [3] Dang, Y., Lin, Q., & Huang, P. (2019). AIOps: Real-world challenges and research innovations. *Proceedings of the 41st IEEE/ACM International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, pp. 4–5.
- [4] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.
- [5] Wu, S., Irsoy, O., Lu, S., Dabrovolski, V., Dredze, M., Gehrmann, S., Kambadur, P., Rosenberg, D., & Mann, G. (2023). BloombergGPT: A large language model for finance. *arXiv preprint arXiv:2303.17564*.
- [6] Dou, Y., Liu, Z., Sun, L., Deng, Y., Peng, H., & Yu, P. S. (2020). Enhancing graph neural network-based fraud detectors against camouflaged fraudsters. *Proceedings of the 29th ACM International Conference on Information and Knowledge Management (CIKM '20)*, pp. 315–324.
- [7] Spirtes, P., Glymour, C., & Scheines, R. (2000). *Causation, Prediction, and Search* (2nd ed.). MIT Press.
- [8] Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems (NeurIPS)*, 30, pp. 4765–4774.
- [9] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.
- [10] Fischer, M. J., Lynch, N. A., & Paterson, M. S. (1985). Impossibility of distributed consensus with one faulty process. *Journal of the ACM*, 32(2), 374–382.

- [11] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33, pp. 9459–9474.
- [12] Bank for International Settlements. (2024). Statistics on payment, clearing and settlement systems in the CPMI countries. BIS Publications.
- [13] Reserve Bank of India. (2024). Annual Report on Digital Payments in India 2023–24. RBI Publications.
- [14] Lin, Q., Zhao, X., Xu, Y., Zhang, H., Kang, Y., Zhang, D., Qiao, B., & Rajmohan, S. (2020). Fast outage analysis of large-scale production clouds with service correlation mining. *Proceedings of the 42nd IEEE/ACM International Conference on Software Engineering (ICSE 2020)*, pp. 955–965.
- [15] Shen, Z., Liu, J., He, Y., Zhang, X., Lin, Z., Liu, H., & Zheng, P. (2022). Towards out-of-distribution sequential event prediction: A causal treatment. *Advances in Neural Information Processing Systems (NeurIPS)*, 35.
- [16] Psaiar, H., & Dustdar, S. (2011). A survey on self-healing systems: Approaches and systems. *Computing*, 91(1), 43–73. <https://doi.org/10.1007/s00607-010-0107-y>
- [17] PCI Security Standards Council. (2022). Payment Card Industry Data Security Standard (PCI DSS) v4.0. PCI SSC Publications.
- [18] Ministry of Electronics and Information Technology, India. (2023). Digital Personal Data Protection Act, 2023. Gazette of India.
- [19] European Parliament. (2024). Regulation on Artificial Intelligence (EU AI Act). Official Journal of the European Union.
- [20] Apache Software Foundation. (2023). Apache Kafka Documentation 3.5. <https://kafka.apache.org/documentation/>
- [21] Lopez-Rojas, E. A., Elmir, A., & Axelsson, S. (2016). PaySim: A financial mobile money simulator for fraud detection. *Proceedings of the 28th European Modeling and Simulation Symposium (EMSS 2016)*, pp. 249–255.

- [22] Bhattacharyya, S., Jha, S., Tharakunnel, K., & Westland, J. C. (2011). Data mining for credit card fraud: A comparative study. *Decision Support Systems*, 50(3), 602–613. <https://doi.org/10.1016/j.dss.2010.08.008>
- [23] Lamport, L. (1978). Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7), 558–565.
- [24] Gartner Inc. (2017). Hype Cycle for IT Operations, 2017. Gartner Research.
- [25] National Payments Corporation of India. (2024). UPI Product Statistics 2023–24. NPCI Publications. <https://www.npci.org.in/what-we-do/upi/product-statistics>